

Database management systems

Lecture 1

Prof. Univ. Dr. Diaconita Vlad

PL/SQL Language

- Is the Procedural Language extension to SQL
- Includes SQL statements for data managing and transaction support
- Includes PL/SQL specific statements
- Is a 3GL language

```
Select first_name into ... -SQL
IF...THEN -PLSQL
    x:=y+1;          -PLSQL
    update employees ... - SQL
ELSE
    update employees ... - SQL
END IF;
```

PL/SQL Language

- Is a block-structured language. The PL/SQL code block helps modularize code by using:
 - Anonymous blocks
 - Procedures and functions
 - Packages
 - Database triggers
- The benefits of using modular program constructs are:
 - Easy maintenance
 - Improved data security and integrity
 - Improved performance
 - Improved code clarity

PL/SQL Language

- Supports defining variables and constants of all the SQL data types
- User defined Types can also be constructed
- Introduces PL/SQL specific data types (e.g. PLS_INTEGER, BOOLEAN)
- Supports the fundamental control structures
- Supports the processing of a data set, row by row

PL/SQL Operators

Operator	Characteristics
+, -, *, /, ** (power operator)	Arithmetic
AND, OR, NOT	Logical
<, >, =, >=, <=, <>, !=	Comparison
BETWEEN ... AND ...	Comparison
IN	Comparison
LIKE	Comparison
IS NULL	Check if NULL
 	Concatenation
@	Connecting at distance (database links)
& or&&	Addressing substitution variables

PL/SQL Block

- Anonymous blocks:
 - Form the basic PL/SQL block structure
 - Initiate PL/SQL processing tasks from applications
 - Can be nested within the executable section of any PL/SQL block

```
[DECLARE      -- Declaration Section (Optional)
  variable declarations; ... ]
BEGIN          -- Executable Section (Mandatory)
  SQL or PL/SQL statements;
[EXCEPTION    -- Exception Section (Optional)
  WHEN exception THEN statements; ]
END;           -- End of Block (Mandatory)
```

Procedures

- Procedures are named PL/SQL blocks that perform a sequence of actions.

```
CREATE PROCEDURE getemp IS  -- header
    emp_id  employees.employee_id%type;
    lname   employees.last_name%type;
BEGIN
    emp_id := 100;
    SELECT last_name INTO lname
    FROM EMPLOYEES
    WHERE employee_id = emp_id;
    DBMS_OUTPUT.PUT_LINE('Last name: ' || lname);
END;
/
```

Functions

- Functions are named PL/SQL blocks that perform a sequence of actions and return a value. A function can be invoked from:
 - Any PL/SQL block
 - A SQL statement (subject to some restrictions)

```
CREATE FUNCTION avg_salary RETURN NUMBER IS
    avg_sal employees.salary%type;
BEGIN
    SELECT AVG(salary) INTO avg_sal
    FROM EMPLOYEES;
    RETURN avg_sal;
END;
/
```


Packages

- PL/SQL packages have a specification and an optional body. Packages group related subprograms together.

```
CREATE PACKAGE emp_pkg IS
    PROCEDURE getemp;
    FUNCTION avg_salary RETURN NUMBER;
END emp_pkg;
/

CREATE PACKAGE BODY emp_pkg IS
    PROCEDURE getemp IS ...
    BEGIN ... END;

    FUNCTION avg_salary RETURN NUMBER IS ...
    BEGIN ... RETURN avg_sal; END;
END emp_pkg;
/
```

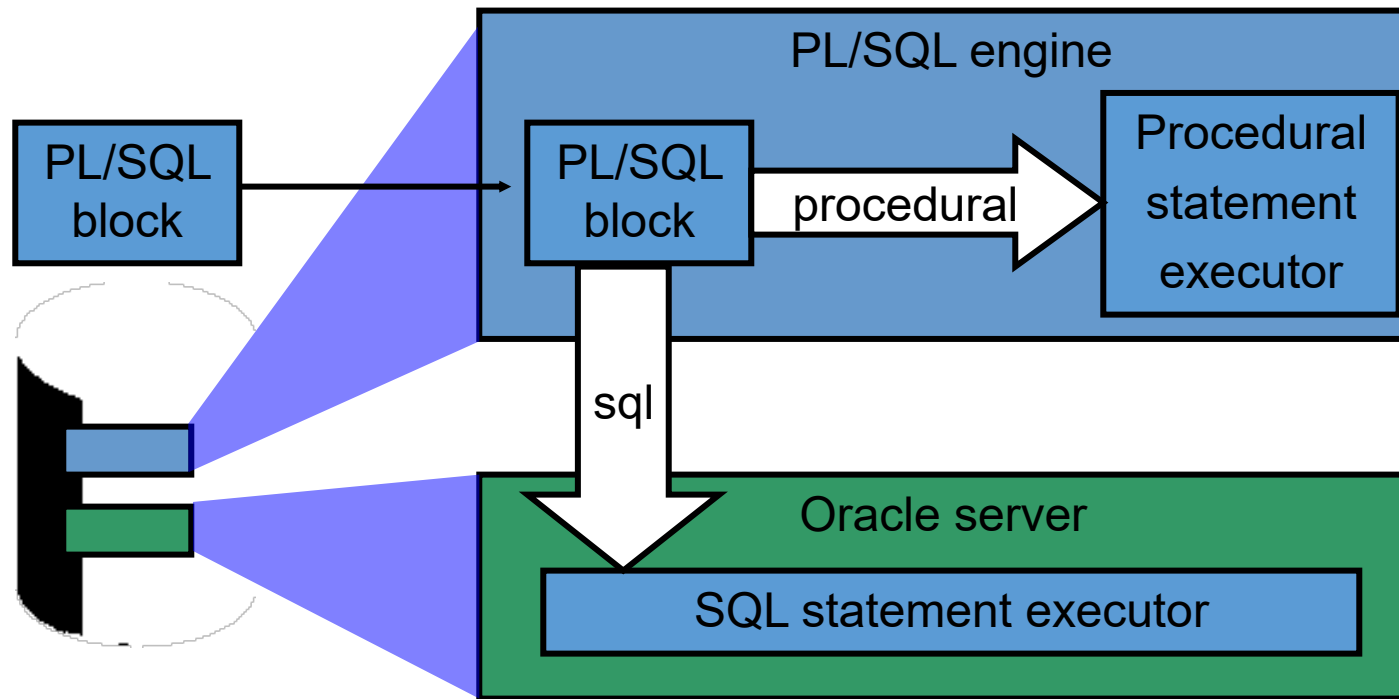
Triggers

- PL/SQL triggers are code blocks that execute when a specified application, database, or table event occurs.
 - Oracle database triggers have a specific structure.
 - Apex dynamic actions (PL/SQL) are similar to triggers

```
CREATE TRIGGER check_salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
DECLARE
    c_min constant number(8,2) := 1000.0;
    c_max constant number(8,2) := 500000.0;
BEGIN
    IF :new.salary > c_max OR
       :new.salary < c_min THEN
        RAISE_APPLICATION_ERROR(-20000,
            'New salary is too small or large');
    END IF;
END;
/
```

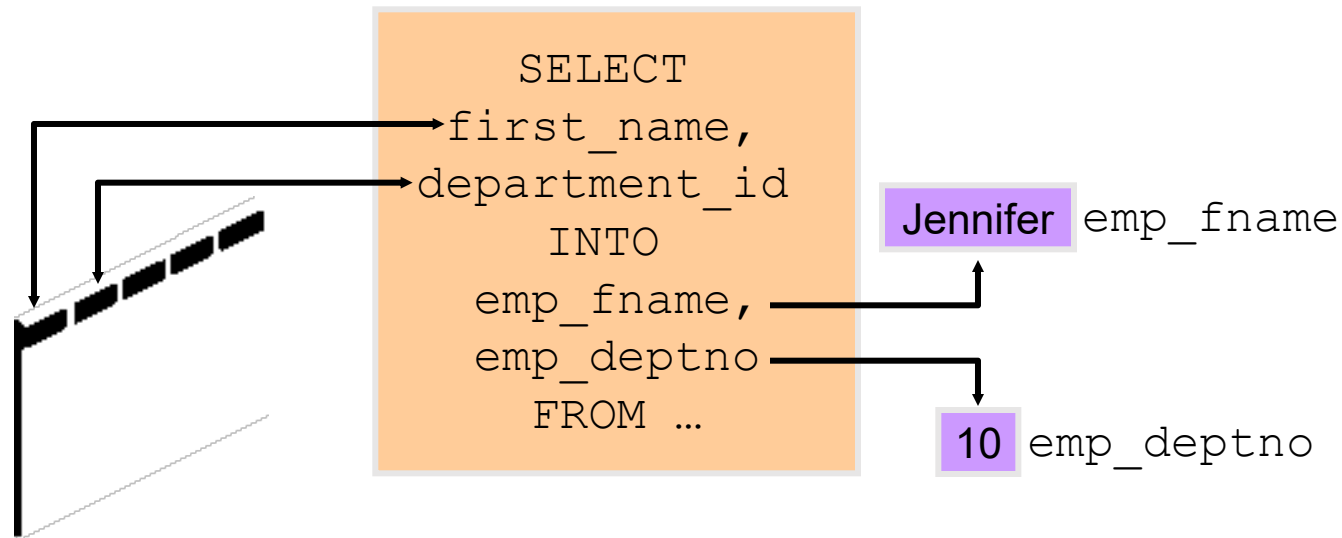
PL/SQL Execution Environment

- The PL/SQL run-time architecture:



Variables

- Variables can be used for:
 - Temporary storage of data
 - Manipulation of stored values
 - Reusability



Identifiers

- Identifiers are used for:
 - Naming a variable
 - Providing conventions for variable names
 - Must start with a letter
 - Can include letters or numbers
 - Can include special characters (such as dollar sign, underscore, and pound sign)
 - Must limit the length to 30 characters
 - Must not be reserved words

Handling Variables in PL/SQL

- Variables are:
 - Declared and initialized in the declarative section
 - Used and assigned new values in the executable section
 - Passed as parameters to PL/SQL subprograms
 - Used to hold the output of a PL/SQL subprogram

Declaring and Initializing PL/SQL Variables

Syntax

```
identifier [CONSTANT] datatype [NOT NULL]  
[:= | DEFAULT expr];
```

Examples

```
DECLARE  
  emp_hiredate    DATE;  
  emp_deptno      NUMBER(2) NOT NULL := 10;  
  location        VARCHAR2(13) := 'Atlanta';  
  c_comm          CONSTANT NUMBER := 1400;
```

Delimiters in String Literals

```
SET SERVEROUTPUT ON
DECLARE
    event VARCHAR2(15);
BEGIN
    event := q'!Father's day!';
    DBMS_OUTPUT.PUT_LINE('3rd Sunday in June is :
    '||event);
    event := q'[Mother's day]';
    DBMS_OUTPUT.PUT_LINE('2nd Sunday in May is :
    '||event);
END;
/
```

3rd Sunday in June is : Father's day
2nd Sunday in May is : Mother's day
PL/SQL procedure successfully completed.

Types of Variables

- PL/SQL variables:
 - Scalar
 - Composite
 - Reference
 - Large object (LOB)
- Non-PL/SQL variables: Bind variables, Substitution

Guidelines for Declaring and Initializing PL/SQL Variables

- Follow naming conventions.
- Use meaningful names for variables.
- Initialize variables designated as `NOT NULL` and `CONSTANT`.
- Initialize variables with the assignment operator (`:=`) or the `DEFAULT` keyword:

```
Myname VARCHAR2(20) := 'John';
```

```
Myname VARCHAR2(20) DEFAULT 'John';
```

- Declare one identifier per line for better readability and code maintenance.

Guidelines for Declaring PL/SQL Variables

- Avoid using column names as identifiers.

```
DECLARE
    employee_id  NUMBER(6);
BEGIN
    SELECT      employee_id
    INTO        employee_id
    FROM        employees
    WHERE       last_name = 'Kochhar';
END;
/
```

- **Use the NOT NULL constraint when the variable must hold a value.**

Base Scalar Data Types

- CHAR [(maximum_length)]
- VARCHAR2 (maximum_length)
- LONG
- LONG RAW
- NUMBER [(precision, scale)]
- BINARY_INTEGER
- PLS_INTEGER
- BOOLEAN
- BINARY_FLOAT
- BINARY_DOUBLE
- DATE
- TIMESTAMP
- TIMESTAMP WITH TIME ZONE
- TIMESTAMP WITH LOCAL TIME ZONE
- INTERVAL YEAR TO MONTH
- INTERVAL DAY TO SECOND

PL/SQL Blocks

```
set serveroutput on
declare
  a number(10,2);
  b number(2) default 7;
  c number(2) default 5;
begin
  a:=max(b,c);
  dbms_output.put_line('a=' || a);
end;
/
```

- ?

```
set serveroutput on
declare
  a number(10,2);
  b number(2) default 7;
  c number(2) default 5;
begin
  a:=greatest(b,c);
  dbms_output.put_line('a=' || a);
end;
/
```

- ?

PL/SQL Blocks

```
set serveroutput on
declare
  a varchar2(3);
  b number(2) default 7;
begin
  a:=decode(b,7, 'YES','NO');
  dbms_output.put_line('a=' |
    |a);
end;
/
```

- ?

```
set serveroutput on
declare
  a varchar2(3);
  b number(2) default 7;
begin
  a:=case when b=7 then 'YES'
    else 'NO' end;
  dbms_output.put_line('a=' || a)
  ;
end;
/
```

- ?

PL/SQL Blocks

```
set serveroutput on
declare
  a number(7);
  b number(7);
begin
  select sum(salary),max(salary) into a,b from
    employees;
  dbms_output.put_line('a=' || a);
  dbms_output.put_line('b=' || b);
end;
/
```

?

```
set serveroutput on
declare
  a number(7);
  b number(7);
begin
  select sum(salary),max(salary) from
    employees;
end;
/
```

?

PL/SQL Blocks

```
set serveroutput on  
begin  
select * from employees;  
end;  
/
```

?

```
set serveroutput on  
declare  
  p_first_name varchar2(20);  
  p_last_name varchar2(20);  
begin  
  select first_name,last_name into p_first_name,  
    p_last_name from employees where  
    employee_id=101;  
  dbms_output.put_line(p_first_name || ' ' ||  
    p_last_name);  
end;  
/
```

?

BINARY_FLOAT and BINARY_DOUBLE

- Represent floating point numbers in IEEE 754 format
- Offer better interoperability and operational speed
- Store values beyond the values that the data type NUMBER can store
- Provide the benefits of closed arithmetic operations and transparent rounding

NUMBER	:	68
BINARY_FLOAT	:	35
BINARY_DOUBLE	:	33

%TYPE Attribute

- The %TYPE attribute
 - Is used to declare a variable according to:
 - A database column definition
 - Another declared variable
 - Is prefixed with:
 - The database table and column
 - The name of the declared variable
 - Examples:

```
emp_lname      employees.last_name%TYPE;
```

```
balance        NUMBER(7,2);
```

```
min_balance    balance%TYPE := 1000;
```

Declaring Boolean Variables

- Only the values `TRUE`, `FALSE`, and `NULL` can be assigned to a Boolean variable.
- Conditional expressions use the logical operators `AND` and `OR` and the unary operator `NOT` to check the variable values.
- The variables always yield `TRUE`, `FALSE`, or `NULL`.
- Arithmetic, character, and date expressions can be used to return a Boolean value.

Bind Variables

- Bind variables are:
 - Created in the environment
 - Also called *host* variables
 - Created with the `VARIABLE` keyword
 - Used in SQL statements and PL/SQL blocks
 - Accessed even after the PL/SQL block is executed
 - Referenced with a preceding colon

```
VARIABLE emp_salary NUMBER
BEGIN
    SELECT salary INTO :emp_salary
    FROM employees WHERE employee_id = 178;
END;
/
PRINT emp_salary
SELECT first_name, last_name FROM employees
WHERE salary=:emp_salary
```

Substitution Variables

- Are used to get user input at run time
- Are referenced within a PL/SQL block with a preceding ampersand
- Are used to avoid hard-coding values that can be obtained at run time

```
VARIABLE emp_salary NUMBER
SET AUTOPRINT ON
DECLARE
    empno NUMBER(6) := &empno;
BEGIN
    SELECT salary INTO :emp_salary
    FROM employees WHERE employee_id = empno;
END;
/
```

Bibliography

- De Elvis Foster, Shripad Godbole, Database Systems: A Pragmatic Approach, Apress, 2012

Database management systems

Lecture 2

Data types

- Scalar types
 - SQL: CHAR, VARCHAR2, NCHAR, NVARCHAR2, RAW, NUMBER, DATE, TIMESTAMP
 - PL/SQL: PLS_INTEGER, BINARY_INTEGER, BOOLEAN
- Composite types
 - Record: user defined, %ROWTYPE
 - **Collections:**
 - Index-by Tables (associative arrays)
 - Nested Tables
 - Variable-size arrays (VARRAY)
- Reference Types: REF_CURSOR
- LOB Types: BFILE, BLOB, CLOB, NCLOB

Composite data types

Record:

TRUE	23-DEC-98	ATLANTA	
------	-----------	---------	---

Collection:

PL/SQL table structure		PL/SQL table structure	
1	SMITH	1	5000
2	JONES	2	2345
3	NANCY	3	12
4	TIM	4	3456
└─ PLS_INTEGER		└─ PLS_INTEGER	
└─ VARCHAR2		└─ NUMBER	

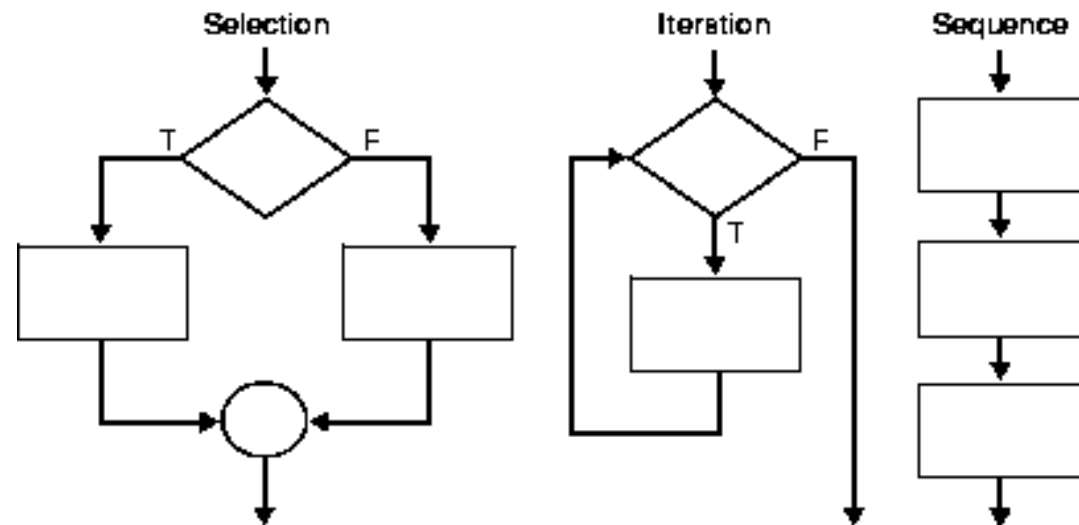
%ROWTYPE

```
SET SERVEROUTPUT ON
DECLARE
  x employees%rowtype;
BEGIN
  select * into x from employees where employee_id=100;
  dbms_output.put_line(x.employee_id);
  dbms_output.put_line(x.first_name);
  dbms_output.put_line(x.last_name);
  dbms_output.put_line(x.email);

END;
/
```

PL/SQL Control Structures

- Sequence
- Decision (conditional, selection): IF, CASE
- Iterative: FOR, WHILE, LOOP



IF-THEN Statement

- IF condition THEN
 - sequence_of_statements1
 - ELSE
 - sequence_of_statements2
 - END IF
- IF condition1 THEN
 - sequence_of_statements1
 - ELSIF condition2 THEN
 - sequence_of_statements2
 -
 - ELSIF conditionN THEN
 - sequence_of_statementsN
 - ELSE
 - sequence_of_statements3
 - END IF;

CASE Statement

- [<<label_name>>]
- CASE selector
- WHEN expression1 THEN sequence_of_statements1;
- WHEN expression2 THEN sequence_of_statements2;
- ...
- WHEN expressionN THEN sequence_of_statementsN;
- [ELSE sequence_of_statementsN+1;]
- END CASE [label_name];

Null Statement

- Passes control to the next statement. In a conditional construct, the NULL statement tells readers that a possibility has been considered, but no action is necessary.
- EXCEPTION
- WHEN ZERO_DIVIDE THEN
- ROLLBACK;
- WHEN VALUE_ERROR THEN
- INSERT INTO errors VALUES ...
- COMMIT;
- WHEN OTHERS THEN
- NULL;
- END;

LOOP and WHILE

- LOOP
 - FETCH c1 INTO ...
 - EXIT WHEN; -- exit loop if condition is true
 - ...
 - END LOOP;
-
- WHILE condition LOOP
 - sequence_of_statements
 - END LOOP;

FOR

- FOR counter IN [REVERSE] lower_bound..higher_bound LOOP
- sequence_of_statements
- END LOOP;

Sequential Control: GOTO and NULL Statements

- **Not commonly used.** GOTO statement cannot branch into an IF statement, CASE statement, LOOP statement, sub-block or from an exception handler into the current block.
- BEGIN
- ...
- GOTO insert_row;
- ...
- <<insert_row>>
- INSERT INTO emp VALUES ...
- END;

Collections

- **Index-by Tables (associative arrays):** Lets you look up elements using arbitrary numbers and strings for subscript values. These are similar to hash tables in other programming languages;
- **Nested Tables:** Use (initially) sequential numbers as subscripts;
- **Variable-size arrays (VARRAY):** Hold a fixed number of elements (although you can change the number of elements at runtime). They use sequential numbers as subscripts.

Comparison

Collection Type	Number of Elements	Subscript Type	Dense or Sparse	Where Created
Associative array (or index-by table)	Unbounded	String or integer	Either	Only in PL/SQL block
Nested table	Unbounded	Integer	Starts dense, can become sparse	Either in PL/SQL block or at schema level
Variable-size array (varray)	Bounded	Integer	Always dense	Either in PL/SQL block or at schema level

Methods (selection)

- EXISTS(n)
- COUNT
- LIMIT(only for NESTED TABLES and VARRAYS)
- FIRST and LAST
- PRIOR and NEXT/PRIOR(N) and NEXT(n)
- EXTEND/EXTEND(n)/EXTEND(n,i) (only for NESTED TABLES)
- TRIM/TRIM(n) (only for NESTED TABLES and VARRAYS)
- DELETE/DELETE(n)/DELETE(n,m) (when n is specified, doesn't work for VARRAYS)

Index-by-tables

DECLARE

```
TYPE EmpTabTyp IS TABLE OF employees%ROWTYPE INDEX BY PLS_INTEGER;  
emp_tab EmpTabTyp;
```

BEGIN

```
SELECT * INTO emp_tab(200) FROM employees WHERE employee_id = 200;  
dbms_output.put_line('Name=' || emp_tab(200).first_name);  
dbms_output.put_line(case when emp_tab.exists(200) then 'exists' else 'does not exist' end);  
dbms_output.put_line(case when emp_tab.exists(1) then 'exists' else 'does not exist' end);  
SELECT * INTO emp_tab(205) FROM employees WHERE employee_id = 205;  
dbms_output.put_line('Name=' || emp_tab(205).first_name);  
dbms_output.put_line('Number of elements=' || emp_tab.count);  
emp_tab(122):=emp_tab(205);  
dbms_output.put_line('Name=' || emp_tab(122).first_name);  
dbms_output.put_line('Number of elements=' || emp_tab.count);  
emp_tab(2005).first_name:='John';  
dbms_output.put_line('Number of elements=' || emp_tab.count);
```

END;

Index-by-tables -> indexed by PLS_INTEGER

```
set serveroutput on
DECLARE
TYPE tab_ind IS TABLE OF employees%ROWTYPE INDEX BY PLS_INTEGER;
t tab_ind;
BEGIN
SELECT * BULK COLLECT INTO t FROM employees WHERE ROWNUM<= 50 order by salary desc;
DBMS_OUTPUT.PUT_LINE('Number of elements '||t.COUNT);
t.delete(t.first);
DBMS_OUTPUT.PUT_LINE('Number of elements '||t.COUNT);
FOR i IN t.FIRST.. t.LAST LOOP
    DBMS_OUTPUT.PUT_LINE(t(i).first_name ||' '||t(i).last_name ||'->'||i);
END LOOP;
END;
/
```

Index-by-tables -> indexed by varchar2

```
SET SERVEROUTPUT ON
DECLARE
TYPE tab_ind IS TABLE OF NUMBER INDEX BY VARCHAR2(1);
t tab_ind;
i varchar2(1);
BEGIN
t('a') := ASCII('a'); t('A') := ASCII('A');
t('b') := ASCII('b'); t('B') := ASCII('B');
t('x') := ASCII('x'); t('X') := ASCII('X');
i := t.FIRST;
WHILE i IS NOT NULL LOOP
    DBMS_OUTPUT.PUT_LINE('t('||i||')='||t(i));
    i := t.NEXT(i);
END LOOP;
END;
/
```

Context Switches and Performance

- PL/SQL statements are run by the PL/SQL statement executor; SQL statements are run by the SQL statement executor
- When the PL/SQL runtime engine encounters a SQL statement, it stops and passes the SQL statement over to the SQL engine
- The SQL engine executes the SQL statement and returns information back to the PL/SQL engine
- This transfer of control is called a context switch, and each one of these switches incurs overhead that slows down the overall performance of your programs

Index-by tables

```
DECLARE

TYPE tab_ind IS TABLE OF employees%ROWTYPE
INDEX BY PLS_INTEGER;

t tab_ind;

BEGIN

SELECT * BULK COLLECT INTO t
FROM employees
WHERE ROWNUM<=50 order by salary desc;

DBMS_OUTPUT.PUT_LINE('Number of elements ' || t.COUNT);

t.delete(t.first);

DBMS_OUTPUT.PUT_LINE('Number of elements ' || t.COUNT);

FOR i IN t.FIRST..t.LAST LOOP

DBMS_OUTPUT.PUT_LINE(t(i).first_name || ' ' || t(i).last_name || '
' || t(i).salary);

END LOOP;

END;
```

```
DECLARE

cursor c is select * FROM employees
WHERE ROWNUM<=50 order by salary desc;

BEGIN

for r in c loop

DBMS_OUTPUT.PUT_LINE(r.first_name || ' ' || r.last_name || '
' || r.salary);

END LOOP;

END;
```

```
set serveroutput on

DECLARE

TYPE tab_ind IS TABLE OF employees%ROWTYPE

INDEX BY PLS_INTEGER;

t tab_ind;

BEGIN

SELECT * BULK COLLECT INTO t

FROM employees minus

SELECT *

FROM employees

WHERE ROWNUM<=50;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

DBMS_OUTPUT.PUT_LINE('Number of elements ' || t.COUNT);

FOR i IN t.FIRST..t.LAST LOOP

    DBMS_OUTPUT.PUT_LINE(t(i).first_name || ' ' || t(i).last_name);

END LOOP;

delete from employees where 1=2;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

FOR i IN t.FIRST..t.LAST loop

    update employees set salary=salary+1 where employee_id=t(i).employee_id;

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

end loop;

END;
```

```
DECLARE

TYPE tab_ind IS TABLE OF employees%ROWTYPE

INDEX BY PLS_INTEGER;

t tab_ind;

BEGIN

SELECT * BULK COLLECT INTO t

FROM employees minus

SELECT *

FROM employees

WHERE ROWNUM<=50;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

DBMS_OUTPUT.PUT_LINE('Number of elements ' || t.COUNT);

FOR i IN t.FIRST..t.LAST LOOP

    DBMS_OUTPUT.PUT_LINE(t(i).first_name || ' ' || t(i).last_name);

END LOOP;

delete from employees where 1=2;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

FORALL i IN t.FIRST..t.LAST

    update employees set salary=salary+1 where
employee id=t(i).employee id;

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);

END;
```

Nested tables

- Within the **database (SQL)**, nested tables are column types that hold sets of values. Oracle stores the rows of a nested table in no particular order.
- When you retrieve a nested table from the database into a PL/SQL variable, the rows are given consecutive subscripts starting at 1. That gives you array-like access to individual rows.
- Nested tables do not have a pre-declared number of elements
- Nested tables might not have consecutive subscripts, while arrays are always dense (have consecutive subscripts).
- **Initially**, nested tables are dense, but they can become **sparse** (have nonconsecutive subscripts). You can delete elements from a nested table using the built-in procedure DELETE. The built-in function NEXT lets you iterate over all the subscripts of a nested table, even if the sequence has gaps.

Nested tables

```
set serveroutput on
DECLARE
TYPE tab_imb IS TABLE OF NUMBER;
t tab_imb := tab_imb(1,20,3,40,5);
t_null tab_imb;
BEGIN
t.EXTEND(5);
FOR i IN 6..10 LOOP
t(i):=t(i-1)+5;
END LOOP;
DBMS_OUTPUT.PUT('Table has ' || t.COUNT || ' elements: ');
FOR i IN t.FIRST..t.LAST LOOP
    DBMS_OUTPUT.PUT(t(i) || ' ');
END LOOP;
DBMS_OUTPUT.NEW_LINE;
t.delete;
IF t IS NULL THEN
    DBMS_OUTPUT.PUT_LINE('Table is null');
```

```
else
    DBMS_OUTPUT.PUT_line('Table has ' || t.COUNT || '
elements');
END IF;
t.extend(2);
t(1):=500;t(2):=700;
DBMS_OUTPUT.PUT_line('Table has ' || t.COUNT || ' elements');
t := t_null;
IF t IS NULL THEN
    DBMS_OUTPUT.PUT_LINE('Table is null');
else
    DBMS_OUTPUT.PUT_line('Table has ' || t.COUNT || '
elements');
END IF;
END;
/
```

Nested tables

```
CREATE TYPE n_t_degrees IS TABLE OF number(8,2);  
CREATE TABLE location_degrees  
( id_location NUMBER(4) PRIMARY KEY,  
  name VARCHAR2(40),  
  degrees n_t_degrees)  
NESTED TABLE degrees STORE AS nested_degrees;  
INSERT INTO location_degrees VALUES (1, 'Constanta', n_t_degrees  
(10,12,14,15));
```

Nested tables

- UPDATE location_degrees SET degrees = n_t_degrees (20,25) WHERE id_location=1;
- SELECT * FROM TABLE(SELECT degrees FROM location_degrees WHERE id_location=1);
- SELECT id_location, name, b.* FROM location_degrees a, TABLE(a.degrees) b;

Varrays

- Have a maximum size, which you specify in its type definition. Its index has a fixed lower bound of 1 and an upper bound.
- Can contain a varying number of elements, from zero (when empty) to the maximum specified in its type definition.
- Are dense, every elements has an index that shows its position
- The DELETE method only works for deleting **all** the elements in the array

Varray

```
DECLARE
TYPE tab_vec IS VARRAY(10) OF NUMBER;
t tab_vec := tab_vec();
BEGIN
FOR i IN 1..10 LOOP
    t.EXTEND;
    t(i):=i;
END LOOP;
DBMS_OUTPUT.PUT('Table has ' || t.COUNT || '
elements: ');
FOR i IN t.FIRST..t.LAST LOOP
    DBMS_OUTPUT.PUT(t(i) || ' ');
END LOOP;
DBMS_OUTPUT.NEW_LINE;
FOR i IN 1..10 LOOP
```

```
    IF i mod 2 = 1 THEN t(i):=null;
    END IF;
END LOOP;
DBMS_OUTPUT.PUT('Table has ' || t.COUNT || '
elements: ');
FOR i IN t.FIRST..t.LAST LOOP
    DBMS_OUTPUT.PUT(nvl(t(i), 0) || ' ');
END LOOP;
DBMS_OUTPUT.NEW_LINE;
--t.delete(1); -- wrong number or types of
arguments in call to 'DELETE'
t.DELETE;
DBMS_OUTPUT.PUT_LINE('Table has '||t.COUNT|| '
elements');
END;
/
```

VARRAYS – Example SQL

- CREATE OR REPLACE TYPE t_vect_projects AS VARRAY(10) OF varchar2(32);
- CREATE TABLE teams
- (team_id NUMBER(4) PRIMARY KEY,
- team_name VARCHAR2(40),
- projects t_vect_projects);
- insert into teams values (1,'Inspire',t_vect_projects('HR Application','Trading Application'));
- insert into teams values (2,'Excel',t_vect_projects('Client Support','HR Application','Trading Application'));
- insert into teams values (3,'Global',t_vect_projects('Advanced Support','Big Data Analytics','Cloud Migration'));

```
select * from teams;
```

Script Output		
Query Result		
All Rows Fetched: 3 in 0,049 seconds		
TEAM_ID	TEAM_NAME	PROJECTS
1	1 Inspire	VLAD_ENG.T_VECT_PROJECTS(HR Application,Trading Application)
2	2 Excel	VLAD_ENG.T_VECT_PROJECTS(Client Support,HR Application,Trading Application)
3	3 Global	VLAD_ENG.T_VECT_PROJECTS(Advanced Support,Big Data Analytics,Cloud Migration)

VARRAYS – Example SQL

- update teams set projects=t_vect_projects('Big Data Analitics','Cloud Migration') where team_id=3;
- select team_id,team_name,b.* from teams a, table(a.projects) b;

```
SQL> select team_id,team_name,b.* from teams a, table(a.projects) b;
TEAM_ID TEAM_NAME COLUMN_VALUE
-----
1 Inspire HR Application
1 Inspire Trading Application
2 Excel Client Support
2 Excel HR Application
2 Excel Trading Application
3 Global Advanced Support
3 Global Big Data Analitics
3 Global Cloud Migration
8 rows selected
```

PL/SQL – Data types

Data Type	Maximum Size in PL/SQL	Maximum Size in SQL
CHAR ^{Foot 1}	32,767 bytes	2,000 bytes
NCHAR ^{Footref 1}	32,767 bytes	2,000 bytes
RAW ^{Footref 1}	32,767 bytes	2,000 bytes
VARCHAR2 ^{Footref 1}	32,767 bytes	4,000 bytes
NVARCHAR2 ^{Footref 1}	32,767 bytes	4,000 bytes
LONG ^{Foot 2}	32,760 bytes	2 gigabytes (GB) - 1
LONG RAW ^{Footref 2}	32,760 bytes	2 GB
BLOB	128 terabytes (TB)	(4 GB - 1) * <i>database_block_size</i>
CLOB	128 TB	(4 GB - 1) * <i>database_block_size</i>
NCLOB	128 TB	(4 GB - 1) * <i>database_block_size</i>

Database management systems

Lecture 5 - Cursors

Managing cursors

- **SQL (Implicit) Cursor**
- **Explicit Cursor**
 - **OPEN-LOOP-FETCH-CLOSE / FOR LOOP**
 - **Cursors with parameters**
 - **The FOR UPDATE clause**
 - **Dynamic cursors**

Managing cursors

- A cursor is a pointer to a pre allocated memory location in the System Global Area (SGA – a memory structure which is shared by all server and background processes). The set of rows the cursor holds is referred to as the active set.
- The attributes that provide information regarding the results of the cursors can be referenced using:
 - SQL%ATTRIBUTE_NAME for implicit cursors. The values are retained until another SQL statement is run;
 - CURSOR_NAME%ATTRIBUTE_NAME for explicit cursor.

Cursor attributes

- All the attributes of the implicit cursors (except SQL%ISOPEN which is always FALSE) are NULL if no SQL statement was previously run
- %ROWCOUNT
 - Has the PLS_INTEGER type
 - Is NULL if no DML statement was previously run
 - Returns the number of rows affected by the previous DML statement
 - Isn't very useful, together with %FOUND and %NOTFOUND, for SELECT statements without BULK COLLECT in the case of implicit cursors
- %FOUND
 - Has the BOOLEAN data type
 - Is NULL if no DML statement was previously run
 - For implicit cursors, is TRUE if the last statement affected/returned at least one row
 - For explicit cursors, is TRUE if the previous FETCH was successful

Cursor attributes

- %NOTFOUND
 - Is NULL if no LMD statement was previously run
 - Has opposite meaning compared to %FOUND
- %BULK_ROWCOUNT
 - A composite attribute designed for use with the FORALL statement.
 - Its *ith* element stores the number of rows processed by the *ith* execution of an UPDATE or DELETE statement.
 - If the *ith* execution affects no rows, %BULK_ROWCOUNT(*i*) returns zero.
- %BULK_EXCEPTIONS
 - Stores information about any exceptions encountered by a FORALL statement that uses the SAVE EXCEPTIONS clause.
 - A loop is required to determine where the exception occurred.
SQL%BULK_EXCEPTIONS(*i*).ERROR_INDEX specifies which iteration of the FORALL loop caused an exception.

Cursor attributes

- %IS_OPEN
 - Has the BOOLEAN type
 - Indicates if a cursor is opened
 - In case of the implicit cursors, it is always FALSE

Implicit Cursors

- Also known as SQL Cursors
- Oracle processes every SQL statement in a PL/SQL block as an implicit cursor, so implicit cursors store information regarding DML statements processing
- Is automatically closed after the statement gets executed

Example

- What will the following block display?

```
set serveroutput on
begin
if SQL%found then dbms_output.put_line(0) ;
else
dbms_output.put_line(1) ;
end if;
end;
/
```

- a) 0
- b) 1
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

- What will the following block display?

```
set serveroutput on
begin
If not SQL%found then
dbms_output.put_line(0) ;
else
dbms_output.put_line(1) ;
end if;
end;
/
```

- a) 0
- b) 1
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

Example

- **What will the following block display?**

```
set serveroutput on
begin
  dbms_output.put_line(decode(SQL%ROWCOUNT,NULL,0,1)) ;
end;
/
```

- a) 0
- b) 1
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

- **What will the following block display?**

```
set serveroutput on
declare
  n pls_integer;
begin
  select decode(sql%rowcount,NULL,1,0) into n
  from dual;
  dbms_output.put_line(n) ;
end;
/
```

- a) 0
- b) 1
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

Example

- What will the following block display?

```
set serveroutput on
begin
dbms_output.put_line(nvl(sql%rowcount,0)) ;
end;
/
```

- a) 0
- b) 1
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

- What will the following block display?

```
set serveroutput on
begin
update employees set salary=salary+10 where
department_id=30;
if SQL%ISOPEN then
dbms_output.put_line('Is opened');
else
dbms_output.put_line('Is not opened');
end if;
end;
/
```

- a) Is opened
- b) Is not opened
- c) Will return a compilation-time error
- d) Will return an exception
- e) NULL

Explicit cursors

- Have names
 - Are unique identifiers in the block
 - Can't appear in a expression
 - They cannot take values
- Are associated with SELECT statements that return more than one row. The INTO clause should not appear in the definition.
- The results returned by a cursor can be processed using
 - OPEN – LOOP – FETCH – CLOSE
 - Using FOR LOOP

Explicit cursors

- If the rows should be processed in an order, the ORDER BY clause should be used
- If an expression appears in the select statement, an alias should be used if the expression will be referenced later
- If we try to open a cursor that is already opened or to close an already closed one, exceptions will be raised

Explicit cursors

- When the cursor is opened, resources get allocated
- The query is processed.
- If FOR UPDATE is included then the rows get blocked
- The cursor is opened and the pointer is positioned before the first row
- At every FETCH the pointer advanced towards the last row (never in reverse)
- The information is read into PL/SQL variables
- When the pointer is positioned after the last row, the looping stops
- When the cursor is closed, resources get released

Examples

```
declare
cursor c is select EMPLOYEE_ID,FIRST_NAME,LAST_NAME,SALARY from EMPLOYEES order by SALARY desc;
r c%rowtype;
begin
open c;
if NOT c%found then dbms_output.put_line('AAA');
else dbms_output.put_line('BBB'); -- c%found is NULL
end if;
loop
fetch c into r;
exit when c%notfound;
dbms_output.put_line(r.FIRST_NAME||' '||r.LAST_NAME||' '||r.SALARY);
end loop;
close c;
end;
/
```

Examples

set serveroutput on

declare

cursor c is select employee_id,first_name,last_name, salary from employees order by salary desc;
r c%rowtype;

begin

open c;

fetch c into r;

while c%found loop

dbms_output.put_line(r.first_name || ' ' || r.last_name || ' ' || r.salary);

fetch c into r;

end loop;

close c;

end;

/

Examples

```
set serveroutput on size 20000
```

```
declare
```

```
  cursor c is select employee_id,first_name,last_name, salary from employees  
order by salary desc;
```

```
begin
```

```
  for r in c loop
```

```
    dbms_output.put_line(r.first_name || ' ' || r.last_name || ' ' || r.salary);
```

```
--inline cursors have no attributes
```

```
  end loop;
```

```
end;
```

```
/
```

Example using FOR

- What will the following block display

```
SET SERVEROUTPUT ON
```

```
declare
```

```
cursor c is select employee_id,first_name,last_name,salary from employees order by salary desc;
```

```
begin
```

```
open c;
```

```
for r in c loop
```

```
dbms_output.put_line(r.first_name || ' ' || r.last_name || ' ' || r.salary);
```

```
end loop;
```

```
close c;
```

```
end;
```

- a) All the employees
- b) One employee
- c) An exception
- d) Two exceptions
- e) Contains a syntax error

Inline cursor

```
begin
```

```
  for r in (select employee_id,first_name,last_name,salary from  
employees order by salary desc) loop
```

```
    dbms_output.put_line(r.first_name||' '||r.last_name||' '||r.salary);
```

```
  end loop;
```

```
end;
```

Cursors with parameters

declare

cursor c(p_id number) is select employee_id,department_id ,first_name,last_name,salary from employees where department_id=p_id order by salary desc;

cursor d is select department_id from employees group by department_id order by count(*) desc;

n number(3);

begin

open d;

fetch d into n;

close d;

for r in c(n) loop

dbms_output.put_line(r.department_id||' '||r.first_name||' '||r.last_name||' '||r.salary);

end loop;

end;

Cursors FOR UPDATE

```
declare
```

```
cursor c(p_id number) is select employee_id,department_id ,first_name,last_name,salary from employees where  
department_id=p_id order by salary desc for update;
```

```
cursor d is select department_id from employees group by department_id order by count(*) desc;
```

```
n number(3);
```

```
begin
```

```
open d;
```

```
fetch d into n;
```

```
close d;
```

```
for r in c(n) loop
```

```
update employees set salary=salary+10 where current of c;
```

```
dbms_output.put_line(r.department_id||' '||r.first_name||' '||r.last_name||' '||r.salary);
```

```
end loop;
```

```
end;
```

```
/
```


Dynamic SQL

REF CURSOR VARIABLES

- Is of REFERENCE type (similar to the POINTER type from C)
- Does not contain any data
- Can be declared as **REF CURSOR** that can be STRONG or WEAK whereas the **SYS_REFCURSOR** predefined type is always WEAK
 - **STRONG**: linked with a table:
 - *TYPE EMD_DATA_T IS REF CURSOR RETURN EMPLOYEES%ROWTYPE;*
 - *X EMP_DATA_T;*
 - **WEAK**: not linked with a table:
 - *X SYS_REFCURSOR;*
- Can be opened for different queries
- Can be used as parameters in subprograms but cannot be declared in the SPECS of a Package
- Can be defined using BIND variables

Dynamic Cursor Example

declare

type t_c is ref cursor return employees%rowtype;

c t_c;

cursor d is select department_id from employees
group by department_id order by count(*) desc;

n number(3);

r employees%rowtype;

begin

open d;

fetch d into n;

close d;

open c for select * from employees where
department_id=n order by salary desc;

loop

fetch c into r;

exit when c%notfound;

dbms_output.put_line(r.department_id || '
' || r.first_name || ' ' || r.last_name || ' ' || r.salary);

end loop;

close c;

end;

Dynamic Cursor Example

```
declare
type t_c is ref cursor return employees%rowtype;
c t_c;

cursor d is select department_id from employees group by
department_id order by count(*) desc;

n number(3);
r employees%rowtype;

begin
open d;
fetch d into n;
close d;

open c for 'select * from employees where
department_id=:1 order by salary desc' using n;
```

```
loop
fetch c into r;
exit when c%notfound;
dbms_output.put_line(r.department_id || ' ' || r.first_name || '
' || r.last_name || ' ' || r.salary);
end loop;
close c;
end;
/
```

Dynamic Cursor Example

```
declare
```

```
type t_c is ref cursor;
```

```
c t_c;
```

```
cursor d is select department_id from employees group by  
department_id order by count(*) desc;
```

```
n number(3);
```

```
r employees%rowtype;
```

```
begin
```

```
open d;
```

```
fetch d into n;
```

```
close d;
```

```
open c for 'select * from employees where department_id=:1  
order by salary desc' using n;
```

```
loop
```

```
fetch c into r;
```

```
exit when c%notfound;
```

```
dbms_output.put_line(r.department_id || ' ' || r.first_name || '  
' || r.last_name || ' ' || r.salary);
```

```
end loop;
```

```
close c;
```

```
end;
```

```
/
```

Dynamic Cursor Example

```
declare
```

```
c SYS_REFCURSOR;
```

```
cursor d is select department_id from employees group by  
department_id order by count(*) desc;
```

```
n number(3);
```

```
r employees%rowtype;
```

```
begin
```

```
open d;
```

```
fetch d into n;
```

```
close d;
```

```
open c for 'select * from employees where department_id=:1  
order by salary desc' using n;
```

```
loop
```

```
fetch c into r;
```

```
exit when c%notfound;
```

```
dbms_output.put_line(r.department_id || ' ' || r.first_name || '  
' || r.last_name || ' ' || r.salary);
```

```
end loop;
```

```
close c;
```

```
end;
```

```
/
```

Dynamic Cursor Example

```
VARIABLE dept_sel REFCURSOR;
```

```
BEGIN
```

```
    OPEN :dept_sel FOR SELECT * FROM DEPARTMENTS;
```

```
END;
```

```
/
```

```
PRINT dept_sel
```

With Functions

```
create or replace function  
syscursor_dep return sys_refcursor is  
  c sys_refcursor;
```

```
  cursor d is select department_id from  
employees group by department_id  
order by count(*) desc;
```

```
  n number(3);
```

```
  r employees%rowtype;
```

```
begin
```

```
  open d;
```

```
  fetch d into n;
```

```
  close d;
```

```
  open c for 'select * from employees  
where department_id=:1 order by  
salary desc' using n;
```

```
  return c;
```

```
end;
```

```
/
```

```
var rc refcursor;
```

```
exec :rc:=syscursor_dep;
```

```
print rc
```

Dynamic Blocks

declare

p varchar2(128);

x varchar2(5);

begin

p:='begin dbms_output.put_line(''Message='' || :e); end;';

x:='ABC';

execute immediate p using x;

end;

Bibliography

- https://docs.oracle.com/cd/B14117_01/appdev.101/b10807/13_elements048.htm
- Gabriela Mihai, Suport de curs SGBD

Database management systems

Exceptions

Errors

- Type of errors:
 - Compilation-time
 - Are detected by the PL/SQL engine
 - Can't be handled in the block because it isn't yet executed
 - Must be corrected by the programmer
 - Run-time
 - In PL/SQL are called exceptions
 - Sometimes, they can occur from design faults, coding mistakes or even hardware failures but in other situations they are part of the design
 - When an exception occurs the block is terminated. The exception has to be handled in the current block (or in an outer block) so the program won't terminate with an error
 - Using exception handlers for error-handling makes programs easier to write and understand, and reduces the likelihood of unhandled exceptions

Exception categories

- Predefined
 - A predefined exception is an internally defined exception that PL/SQL has given a name.
- Non-predefined
 - The runtime system raises internally defined exceptions implicitly (automatically).
 - An internally defined exception always has an error code but does not have a name unless PL/SQL gives it one or you give it one.
- User-defined
 - Users can define exceptions in the declarative part of any PL/SQL anonymous block, subprogram, or package.
 - These exceptions must be raised explicitly

See: https://docs.oracle.com/cd/B19306_01/appdev.102/b14261.pdf
(chapter 10)

Exception handling

```
BEGIN
  statements
EXCEPTION
  WHEN ex_name_1 THEN statements_1           -- Exception handler
  WHEN ex_name_2 OR ex_name_3 THEN statements_2 -- Exception handler
  WHEN OTHERS THEN statements_3               -- Exception handler
END;
```

Error handling

```
set serveroutput on
declare
  fn employees.first_name%type;
begin
  select first_name into fn from employees where employee_id=200;
  select first_name into fn from employees where employee_id=300;
exception
  when no_data_found then
    dbms_output.put_line('The employee wasn't found!');
end;
```

Error handling

```
declare
  fn employees.first_name%type;
  q number(2);
begin
  q:=1;
  select first_name into fn from employees where employee_id=200;
  q:=2;
  select first_name into fn from employees where employee_id=300;
exception
  when no_data_found then
    dbms_output.put_line('The employee wasn't found at query ' || q);
end;
```

Example

- What will the following block return

```
1) declare
2)   fn varchar2(128);
3)   q number(2);
4)   begin
5)     for i in 1..500 loop
6)       if (i mod 2=1) then
7)         i:=i+1;
8)       end if;
9)       select first_name || ' ' || last_name into fn from employees where employee_id=i;
10)      dbms_output.put_line(fn);
11)    end loop;
12)  end;
```

- a) An exception at the 5th line
- b) An exception at the 6th line
- c) An exception at the 7th line
- d) A compilation-time error
- e) The block will run successful

Example

- What will the following block return

- 1) declare
- 2) fn varchar2(128);
- 3) q number(2);
- 4) begin
- 5) for i in 1..500 loop
- 6) if (i mod 2=1) then
- 7) i:=i+1;
- 8) end if;
- 9) select first_name||' '||last_name into fn from employees where employee_id=i;
- 10) dbms_output.put_line(fn);
- 11) end loop;
- 12) end;

- a) An exception at the 5th line
- b) An exception at the 6th line
- c) An exception at the 7th line
- d) A compilation-time error
- e) The block will run successful

```
SQL>
declare
  fn varchar2(128);
  q number(2);
begin
  for i in 1..500 loop
    if (i mod 2=1) then
      i:=i+1;
    end if;
    select first_name||' '||last_name into fn from employees where employee_id=i;
    dbms_output.put_line(fn);
  end loop;
end;
ORA-06550: line 7, column 1:
PLS-00363: expression 'I' cannot be used as an assignment target
ORA-06550: line 7, column 1:
PL/SQL: Statement ignored
```

Example

- What will the following block return

```
1) Set serveroutput on
2) declare
3) fn varchar2(128);
4) q number(2);
5) begin
6) for i in 1..500 loop
7)   begin
8)     select first_name || ' ' || last_name || ' ' || i into fn from employees where employee_id=i;
9)     exception when
10)      no_data_found then null;
11)   end;
12)   dbms_output.put_line(fn);
13) end loop;
14) end;
```

- a) An exception at the 6th line
- b) An exception at the 7th line
- c) An error
- d) Will display once, all the employees that exist with IDs between 1 and 500
- e) Can display one employee more than once

Example

- What will the following block return

```
1) Set serveroutput on
2) declare
3)   fn varchar2(128);
4)   q number(2);
5) begin
6)   for i in 1..500 loop
7)     begin
8)       select first_name || ' ' || last_name || ' ' || i into fn from employees where employee_id=i;
9)       dbms_output.put_line(fn);
10)    exception when no_data_found then null;
11)  end;
12) end loop;
13) end;
```

- a) An exception at the 6th line
- b) An exception at the 7th line
- c) A compilation-time error
- d) Will display once, all the employees that exist with IDs between 1 and 500
- e) Can display one employee more than once

Error Code and Error Message Retrieval

- The error code can be retrieved using the SQLCODE function
 - For an internally defined exception, the numeric code is the number of the associated Oracle Database error (negative, except for NO_DATA_FOUND : +100).
 - For a user-defined exception, the numeric code is either +1 (default) or the error code associated with the exception by the EXCEPTION_INIT pragma.
- The error message can be retrieved using the SQLERRM function
- Outside an exception handler, or if the value of error_code is zero, SQLERRM returns ORA-0000.
- A SQL statement cannot invoke SQLCODE or SQLERRM.

Example

```
create table occurred_exc (exc_user varchar2(32), exc_date date, exc_code  
number(32), exc_message varchar2(256));
```

```
begin
```

```
raise no_data_found;
```

```
exception
```

```
when others then
```

```
insert into occurred_exc values(user,sysdate,SQLCODE,SQLERRM);
```

```
end;
```

- **What will the block return?**
 - a) **An exception**
 - b) **A compilation-time error**
 - c) **Will execute successfully**

Example

```
declare
  c number(10);
  s varchar2(5);
begin
  raise no_data_found;
exception
  when others then
    c:=SQLCODE;
    s:=SQLERRM;
    insert into occurred_exc values(user,sysdate,c,s);
    commit;
end;
```

- What will be the result?
 - a) Will raise an exception that is handled
 - b) Will return an exception that is not handled
 - c) Will return an error
 - d) Will execute successfully

Example

- `set serveroutput on`
- `declare`
- `c number(10);`
- `s varchar2(5);`
- `begin`
- `raise no_data_found;`
- `exception`
- `when others then`
- `c:=SQLCODE;`
- `begin`
- `dbms_output.put_line('Er1=' || SQLERRM);`
- `s:=SQLERRM;`
- `dbms_output.put_line('Er2=' || s);`
- `exception`
- `when others then`
- `dbms_output.put_line('Er3=' || SQLERRM);`
- `insert into occurred_exc values(user,sysdate,c+1,s);`
- `end;`
- `end;`

Example

- set serveroutput on
- declare
- c number(10);
- s varchar2(5);
- begin
- raise no_data_found;
- exception
- when others then
- c:=SQLCODE;
- begin
- dbms_output.put_line('Er1=' || SQLERRM);
- s:=SQLERRM;
- dbms_output.put_line('Er2=' || s);
- exception
- when others then
- dbms_output.put_line('Er3=' || SQLERRM);
- insert into occurred_exc values(user,sysdate,c+1,s);
- end;
- end;

```
SQL>
Er1=ORA-01403: no data found
Er3=ORA-06502: PL/SQL: numeric or value error: character string buffer too small
ORA-01403: no data found
PL/SQL procedure successfully completed
```

```
SQL> select * from occurred_exc;
```

EXC_USER	EXC_DATE	EXC_CODE	EXC_MESSAGE
VLAD_ENG	7/4/2018 8:	101	

```
SQL> |
```


Example

```
Set serveroutput on
declare
cursor c is select first_name from
employees;
r c%rowtype;
begin
  begin
    fetch c into r;
    dbms_output.put_line('Name=' || r.first_n
ame );
    exception when no_data_found then
      dbms_output.put_line('A');
  end;
```

```
dbms_output.put_line('B');
exception
when others then
  dbms_output.put_line('C');
end;
```

What will the block display?

- a) Name of a client
- b) A B
- c) B C
- d) An exception because the cursor is not opened
- e) C

Non predefined exception

- To handle error conditions (typically ORA- messages) that have no predefined name, you must use the OTHERS handler or the pragma EXCEPTION_INIT. A pragma is a compiler directive that is processed at compile time, not at run time.
- In PL/SQL, the pragma EXCEPTION_INIT tells the compiler to associate an exception name with an Oracle error number. That lets you refer to any internal exception by name and to write a specific handler for it. When you see an error stack, or sequence of error messages, the one on top is the one that you can trap and handle.
- You code the pragma EXCEPTION_INIT in the declarative part of a PL/SQL block, subprogram, or package using the syntax
 - PRAGMA EXCEPTION_INIT(exception_name, -Oracle_error_number);
- where exception_name is the name of a previously declared exception and the number is a negative value corresponding to an ORA- error number. The pragma must appear somewhere after the exception declaration in the same declarative section, as shown in the following example:

set SERVEROUTPUT on

declare

is_not_null exception;

pragma exception_init(is_not_null,-01400);

begin

insert into employees(employee_id,first_name) values (1001, 'John');

exception

when is_not_null then

dbms_output.put_line('Not enough values');

end;

/

User defined exceptions

- The procedure `RAISE_APPLICATION_ERROR` lets you issue user-defined ORA- error messages from stored subprograms. That way, you can report errors to your application and avoid returning unhandled exceptions.
 - `raise_application_error(error_number, message[, {TRUE | FALSE}]);`
- where `error_number` is a negative integer in the range -20000 .. -20999 and `message` is a character string up to 2048 bytes long. If the optional third parameter is `TRUE`, the error is placed on the stack of previous errors. If the parameter is `FALSE` (the default), the error replaces all previous errors. `RAISE_APPLICATION_ERROR` is part of package `DBMS_STANDARD`, and as with package `STANDARD`, you do not need to qualify references to it.
- An application can call `raise_application_error` only from an executing stored subprogram (or method). When called, `raise_application_error` ends the subprogram and returns a user-defined error number and message to the application. The error number and message can be trapped like any Oracle error.
- In the following example, we call `raise_application_error` if no salary was increased:

```
BEGIN
  UPDATE employees SET salary = salary + 100 WHERE employee_id = 1000;
  if SQL%NOTFOUND then
    raise_application_error(-20101, 'The employee does not exist');
  end if;
END;
```

Bibliography

- https://docs.oracle.com/cd/B14117_01/appdev.101/b10807/13_elements048.htm

Database management systems

Lecture 8 – Stored Subprograms

Subprograms

- Are units that perform tasks
- Can be combined to form more complex programs
- Can be created:
 - At schema level – standalone subprograms
 - Inside a package – packed subprograms
 - Inside a PL/SQL block

Subprograms

- Parts:
 - Declarative part
 - Executable part
 - Exception handling
- Types
 - Procedures – perform actions, can return values through OUT or IN OUT parameters
 - Functions – usually return a single value. If the requirements are met, can be used in SQL statements like single-row functions
- Parameters
 - IN
 - OUT
 - IN OUT

Advantages

- SGA:
 - Shared pool
 - DB buffer cache
 - Redo log buffer
 - Large pool
 - Java pool
- The code is easier to maintain.
- Updates required by multiple applications can be done once
- The code is re-usable after the procedure is compiled successfully
- Contributed to applications' security
- Improves performance
 - In the data dictionary, exists both **source code** and **p-code (bytecode)**.
 - PL/SQL reads the compiled version of a procedure or package into the shared pool buffer of the SGA when it is invoked and not currently in the SGA.
 - Can be used by multiple users at once
 - *Least recently used* algorithm is used to manage the shared pool

Procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name  
[(parameter_name [IN | OUT | IN OUT] type [, ...])]  
{IS | AS}  
BEGIN  
    < procedure_body >  
END [procedure_name];
```

Passing parameters

- ***get_orders(p_client number, p_category varchar2 default null, p_year number default null)***
 - Positional notation – `get_orders(a,b,c)`, `get_orders(101,'hardware1',2015);`
 - Named notation – `get_orders(p_year=>2015, p_client=>101, p_category=>'hardware1')`
 - Exec `get_orders(p_client=>101);`
 - Mixed notation
 - `get_orders(101,p_year=>2015)` – OK,
 - `get_orders(p_client=>a,b,c)` – NOT OK

Executing a procedure

- Using EXECUTE/EXEC
 - It's an SQL Plus statement that encloses the block in an anonymous block (BEGIN...END). Usually requires less memory compared to CALL
- Using CALL
 - It's a SQL statement and it can only be used with SQL types. Can inhibit the propagation of exceptions.
- From another PL/SQL Block

Executing a procedure

```
CREATE OR REPLACE PROCEDURE get_total_orders
(p_id_emp IN ORDERS.CUSTOMER_ID%type)
IS
v_val number;
BEGIN
Select sum(unit_price*quantity) into v_val from order_items i,orders o where i.order_id=o.order_id and o.CUSTOMER_ID=p_id_emp;
Dbms_output.put_line('Total value= ' || v_val);
END;
/
```

```
SQL> set serveroutput on
SQL> CALL get_total_orders(109);
SQL> EXECUTE get_total_orders(109);

begin
get_total_orders(109);
end;
/
```

Executing a procedure

```
CREATE OR REPLACE PROCEDURE get_total_orders_o
```

```
(p_id_emp IN ORDERS.CUSTOMER_ID%type, p_val out number)
```

```
IS
```

```
BEGIN
```

```
Select nvl(sum(unit_price*quantity),0) into p_val from order_items i,orders o where  
i.order_id=o.order_id and o.CUSTOMER_ID=p_id_emp;
```

```
END;
```

```
/ declare n number;
```

```
VARIABLE n NUMBER
```

```
begin
```

```
VARIABLE n NUMBER
```

```
call
```

```
get_total_orders_o(109,n); EXECUTE
```

```
get_total_orders_o(109,:n)
```

```
dbms_output.put_line('Value='||n);
```

```
get_total_orders_o(109,:n) print n
```

```
print n
```

```
end;
```

Executing a procedure

```
CREATE OR REPLACE PROCEDURE test_exc  
IS  
BEGIN  
  raise too_many_rows;  
END;  
/
```

Which displays the exception's message?

```
call test_exc()  
execute test_exc
```

Other clauses - AUTHID

- AUTHID DEFINER – execute with the owner's permissions (default)
- AUTHID CURRENT_USER – executes with the current user's permission

*CREATE OR REPLACE PROCEDURE test_authid(n out number) authid
current_user*

IS

BEGIN

select count() into n from employees;*

END;

Other clauses - AUTONOMOUS_TRANSACTION

- The procedure is autonomous
- The main transaction gets suspended
- COMMIT or ROLLBACK must be specified in the procedure

What will return?

- CREATE OR REPLACE PROCEDURE test_at(p_id number)
- IS
- PRAGMA AUTONOMOUS_TRANSACTION;
- BEGIN
- update employees e set e.salary=e.salary+10 where e.employee_id=p_id;
- END;
- /

- declare
- s number;
- begin
- select salary into s from employees where employee_id=120;
- dbms_output.put_line('Before=' || s);
- test_at(120);
- select salary into s from employees where employee_id=120;
- dbms_output.put_line('After=' || s);
- end;
- /

Functions

- Are named PL/SQL blocks that must return a value
- Are stored as schema objects
- Can be used in SQL statements if:
 - has only IN parameters
 - returns valid SQL data types
 - doesn't contain COMMIT or ROLLBACK
 - used in SELECT statements, they can't contain INSERT, UPDATE or DELETE
 - used in UPDATE or DELETE on a table, they can't contain DML statements on the same table
 - they don't contain DDL or DCL statements (using execute immediate)

Functions

```
CREATE [OR REPLACE] FUNCTION function_name  
[(parameter_name [IN | OUT | IN OUT] type [, ...])]  
RETURN return_datatype  
{IS | AS}  
BEGIN  
    < function_body >  
END [function_name];
```

Example

- Given the function:

```
CREATE OR REPLACE FUNCTION check_salary
(p_id employees.employee_id%type, p_sal
number)
RETURN Boolean
IS
v_salary employees.salary%type;
BEGIN
SELECT salary into v_salary from employees
where employee_id=p_id;
IF p_sal > v_salary then
return true;
ELSE
```

```
return false;
end if;
EXCEPTION
WHEN no_data_found THEN
return NULL;
end;
/
```

- What will ***select check_salary(109,5000) from dual*** return?
 - a) An exception
 - b) A compilation-time error
 - c) Always NULL
 - d) Just TRUE or FALSE

Example

- Given the function:

```
CREATE OR REPLACE FUNCTION check_salary_n
(p_id employees.employee_id%type, p_sal
number)
RETURN number
IS
v_salary employees.salary%type;
BEGIN
SELECT salary into v_salary from employees
where employee_id=p_id;
IF p_sal > v_salary then
return 1;
ELSE
```

```
return -1;
```

```
end if;
```

```
EXCEPTION
```

```
WHEN no_data_found THEN
```

```
return NULL;
```

```
end;
```

What will ***select check_salary_n(109,5000) from dual*** return?

- a) An exception
- b) A compilation-time error
- c) Always NULL
- d) -1, 1 or NULL

Example

- Given the function:

```
CREATE OR REPLACE FUNCTION check_salary_n
(p_id employees.employee_id%type, p_sal
number)
RETURN number
IS
v_salary employees.salary%type;
BEGIN
SELECT salary into v_salary from employees
where employee_id=p_id;
IF p_sal > v_salary then
return 1;
ELSE
```

```
return -1;
end if;
EXCEPTION
WHEN no_data_found THEN
return NULL;
end;
```

What will ***select first_name, check_salary_n(employee_id, 5000) from employees*** return?

- a) An exception
- b) A compilation-time error
- c) Always NULL
- d) -1, 1 or NULL for each employee

Example

- Given the same function, which of the following will work:
 - delete from employees where check_salary_n(employee_id,5000)=-1;
 - select * from orders where check_salary_n(sales_rep_id,5000)=-1;
 - delete from orders where check_salary_n(sales_rep_id,5000)=-1;

Will it work?

- create or replace function get_dept_emps(p_dep in number) return sys_refcursor is
- dep sys_refcursor;
- begin
- open dep for 'select first_name,last_name from employees where department_id = :1' using p_dep;
- return dep;
- end;
- /

- variable x refcursor
- exec :x:=get_dept_emps(80)
- print x

Will it work?

```
declare
  x sys_refcursor;
begin
  x:=get_dept_emps(80);
  for r in x loop
    dbms_output.put_line(r.first_name || ' ' || r.last_name);
  end loop;
end;
/
```

Will it work?

```
declare
x sys_refcursor;
f_n varchar2(25);
f_l varchar2(25);
begin
x:=get_dept_emps(80);
loop
fetch x into f_n,f_l;
exit when x%notfound;
  dbms_output.put_line(f_n||' '||f_l);
end loop;
end;
/
```

Recursion in PL/SQL

```
CREATE OR REPLACE FUNCTION fibonacci(n NUMBER)
```

```
RETURN NUMBER
```

RESULT_CACHE -- If the cache contains the result from a previous call to the function with the same parameter values, the system returns the cached result to the invoker and does not reexecute the function body.

```
IS
```

```
BEGIN
```

```
IF n = 0 THEN
```

```
RETURN 0;
```

```
elsif n = 1 then return 1;
```

```
ELSE
```

```
RETURN fibonacci(n - 1) + fibonacci(n - 2);
```

```
END IF;
```

```
END;
```

```
/
```

```
variable x number
```

```
exec :x:=fibonacci(10)
```

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}	F_{20}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	6765

Bibliography

- https://docs.oracle.com/cd/B14117_01/appdev.101/b10807/toc.htm
- Gabriela Mihai, Suport de curs SGBD